

UF01 - Acceso a ficheros:

Ejercicios

Ficheros de texto y binarios

Ejercicio 1: Lectura de un fichero de texto

Crea un programa Java que lea el contenido de un fichero de texto llamado **alumnos.txt** situado en la carpeta del proyecto. El programa deberá utilizar **FileReader** y **BufferedReader** para leer el fichero **línea a línea** y mostrar por consola todo su contenido.

Además, deberá mostrar al final:

- El número total de líneas del fichero.
- El número total de caracteres leídos, sin contar los saltos de línea.

El programa deberá controlar mediante **try-with-resources** las posibles excepciones relacionadas con la apertura y lectura del fichero.

Ejercicio 2: Crear y escribir un fichero de texto

Crea un programa Java que solicite al usuario los datos de **5 alumnos**. Para cada alumno se deberá introducir su nombre y su nota. El programa deberá guardar la información en un fichero llamado **notas.txt**, utilizando **FileWriter** y **BufferedWriter**.

Cada alumno deberá ocupar una línea con el siguiente formato:

Nombre: Ana - Nota: 8.5

El fichero deberá abrirse de forma que **se borre su contenido anterior** antes de escribir los nuevos datos. Una vez finalizada la escritura, el programa deberá informar de que el fichero se ha creado/actualizado correctamente. El programa deberá controlar las posibles excepciones.

Ejercicio 3: Añadir información al final de un fichero

Crea un programa Java que permita registrar incidencias de un sistema informático. El programa deberá solicitar al usuario una descripción de la incidencia y guardarla en un fichero llamado **incidencias.txt**.

Cada vez que se ejecute el programa, la nueva incidencia deberá añadirse **al final del fichero**, sin eliminar las incidencias almacenadas anteriormente. Para realizar la

escritura se deberán utilizar **FileWriter** y **BufferedWriter**. Cada incidencia deberá almacenarse en una línea independiente e incluir un número de incidencia. Por ejemplo:

```
Incidencia 1: El servidor no responde
Incidencia 2: Error al conectar con la base de datos
Incidencia 3: El servicio web está detenido
```

El programa deberá calcular automáticamente el número que corresponde a la nueva incidencia a partir de las que ya existen en el fichero. Se deberán controlar las posibles excepciones de entrada/salida.

Ejercicio 4: Escritura y lectura de un fichero binario

Crea un programa Java que permita almacenar y recuperar las edades de un grupo de personas mediante un **fichero binario** llamado **edades.dat**.

El programa deberá:

1. Solicitar al usuario el nombre y la edad de **3 personas**.
2. Escribir los tres nombres y las tres edades en el fichero utilizando **FileOutputStream** y **DataInputStream**.
3. Cerrar el fichero.
4. Volver a abrir el mismo fichero utilizando **FileInputStream** y **DataInputStream**.
5. Leer los tres nombres y las tres edades almacenadas.
6. Mostrar por consola los nombres y edades recuperados y calcular su media.

Recuerda que el fichero deberá almacenarse en **formato binario**, por lo que no se deberá utilizar ningún **Writer** ni **Reader**.

Ficheros CSV con OpenCSV

Ejercicio 5: Leer archivos CSV con distintos delimitadores

Dispones de dos archivos CSV que contienen exactamente los mismos datos de alumnos, pero utilizan diferentes delimitadores:

- a) **alumnos_comas.csv** utiliza **,** como delimitador.
- b) **alumnos_puntoycoma.csv** utiliza **;** como delimitador.

Los archivos contienen las siguientes columnas:

nombre,apellido,dirección,edad

Crea un programa Java **que utilice OpenCSV** para leer ambos archivos y mostrar su contenido por pantalla. El programa deberá:

- Abrir y leer **alumnos_comas.csv** utilizando las clases de OpenCSV.
- Abrir y leer **alumnos_puntoycoma.csv**, indicando a OpenCSV que el delimitador utilizado es ;.
- Mostrar por pantalla los datos de ambos archivos identificando claramente qué datos proceden de cada archivo.

Importante: no se permite realizar el análisis de las líneas mediante **split()**. La lectura y separación de los campos deberá realizarse mediante las funcionalidades proporcionadas por OpenCSV.

Ejercicio 6: Crear un archivo CSV a partir de datos introducidos por teclado

Crea un programa Java que permita registrar los datos de 3 alumnos introducidos por teclado. Para cada alumno se deberá solicitar:

- Nombre.
- Apellido.
- Edad.
- Nota.

El programa deberá utilizar OpenCSV para crear un archivo llamado **alumnos.csv**. El archivo deberá utilizar ' , ' como delimitador y deberá contener una primera línea con los nombres de las columnas:

nombre,apellido,edad,nota

A continuación, se almacenarán los datos introducidos por el usuario. Por ejemplo, si se introducen los datos de tres alumnos, el fichero podría quedar así:

```
nombre,apellido,edad,nota
Ana,García,20,8.5
Carlos,Martínez,21,7.2
Laura,Pérez,19,9.1
```

El programa deberá utilizar las clases y métodos proporcionados por OpenCSV tanto para crear el archivo como para escribir los registros. No se deberá construir manualmente cada línea concatenando los campos mediante **+**, deberá utilizarse la función correspondiente que aparece en el documento de teoría.

Ejercicio 7: Convertir un CSV de ' , ' a ' ; '

Dispones de un archivo llamado **productos.csv** que contiene información sobre productos y utiliza ' , ' como delimitador. Crea un programa Java que utilice **OpenCSV** y que escriba en un nuevo archivo **productos_puntoycoma.csv** exactamente los mismos datos, pero utilizando ' ; ' como delimitador.

El programa deberá **utilizar exclusivamente las funcionalidades de OpenCSV** para leer y escribir los registros. La idea es que el programa lea los campos como datos independientes y posteriormente los vuelva a escribir utilizando un delimitador diferente sin utilizar el operador de concatenación "+".

Lectura, Modificación y Escritura de XML en Java

Ejercicio 8: Lectura de una biblioteca XML

Dispones de un fichero XML llamado **libros.xml** que contiene información sobre varios libros. Crea un programa Java que:

- Lea el fichero **libros.xml** y lo transforme en un objeto **Document**.
- Muestre por pantalla el título, autor y precio de cada libro.
- Muestre finalmente el número total de libros.

El programa deberá producir una salida similar a:

```
Libro: El principito
Autor: Antoine de Saint-Exupéry
Precio: 12.50 €

Libro: 1984
Autor: George Orwell
Precio: 15.90 €

Libro: Don Quijote de la Mancha
Autor: Miguel de Cervantes
Precio: 18.00 €

Número de libros: 3
```

Ejercicio 9: Modificar y eliminar datos de alumnos

Dispones del fichero `alumnos.xml`.

Crea un programa Java que:

- Lea el fichero XML y lo convierta en un objeto **Document**.
- Busque al alumno cuyo **id** sea 2 y modifique su nota final, estableciéndola en 9.0.
- Busque al alumno cuyo **id** sea 3 y lo elimine del XML.
- Guarde el resultado en un nuevo fichero llamado `alumnos_modificados.xml`.
- El resto de la información deberá mantenerse sin cambios.

Ejercicio 10: Crea un catalogo de productos desde 0

En este caso queremos crear un catalogo con 2 productos en XML desde 0 (no tengo nada en formato XML que modificar, voy a crear un nuevo objeto y a escribirlo en un XML). Para ello primero creamos el objeto **Document** vacío dentro de nuestra función con estas líneas:

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document document = builder.newDocument();
```

Y añadimos un primer nodo "productos":

```
Element productos = document.createElement("productos");
document.appendChild(productos);
```

A partir de esta estructura inicial, crear el archivo `productos.xml` que contenga lo siguiente:

```
<productos>
  <producto codigo="P001">
    <nombre>Teclado mecánico</nombre>
    <precio>59.90</precio>
    <stock>25</stock>
  </producto>
```

```
<producto codigo="P002">
  <nombre>Ratón inalámbrico</nombre>
  <precio>24.50</precio>
  <stock>40</stock>
</producto>
</productos>
```